

Enterprise Visual Intelligence API - Technical Context

Document version: June 2026

Scope: De
labeled exp

1. Project purpose

What this application does

Currently implemented: A small FastAPI service that accepts an operational image via `POST /analyze`, runs multimodal vision analysis through OpenAI, classifies the scene type, grounds the result against local markdown policy documents, assembles a structured operational report, persists it as JSON, and exposes retrieval via `GET /reports/{report\_id}`.

Business / enterprise problem it solves

It targets operational visual review where humans would otherwise inspect photos and compare them to policy:

- warehouse_scene - Walkway obstructions, stacking, spills

- retail_sh

The README frames this as simulating enterprise AI workflows: multimodal ingestion, structured extraction, policy context, escalation-aware reporting, and persistence.

What type of AI system it is

Currently implemented: A vision-language model (VLM) pipeline with rule-based post-processing, not an agentic system.

Layer | Mechanism

Perception /

Best described as: structured VLM extraction + policy-grounded report assembly.

2. Current architecture

High-level flow

Client -> FastAPI (main.py) -> Routes (/analyze, /health, /reports, /) -> Services (vision, report, context, storage) -> OpenAI API / context_docs / reports JSON files

Main application entry point

- Entry: app/main.py - creates FastAPI(), registers routers, defines GET /

- Run loca

API routes / endpoints

Method | Path | Module | Behavior

GET | / | a

OpenAPI/Swagger at /docs when running.

Service layer

- vision_service.py - OpenAI client, multimodal call, JSON parse, VisualFacts validation

- context_

Schema / model layer

app/schemas.py (Pydantic v2):

- VisualFacts - VLM output contract

- Issue - t

Storage / persistence layer

- Reports: reports/{uuid}.json

- Policy co

Configuration layer

app/config.py - pydantic_settings.BaseSettings:

- app_name, openai_api_key, openai_model (default gpt-4o-mini)

- Loads fr

Test structure

- tests/test_health.py - GET /health

- tests/tes

pytest.ini sets pythonpath = .

Docker / deployment setup

- Dockerfile - Python 3.11-slim, pip install, expose 8000

- docker-c

Not present: Kubernetes, Terraform, cloud IaC, GitHub Actions.

3. Current implemented tech stack

Category | Choice

Backend |

4. End-to-end workflow: POST /analyze

Sequence (implemented)

1. Client sends multipart file upload to POST /analyze

2. validate_

Escalation rules (report_service.py)

- critical: risks exist AND "critical" in policy text AND risk contains blocked/exit/walkway/fire/obstruction

- high: risk

Image handling notes

- Data URL always uses image/png label even for JPEG uploads

- No file s

5. Important files

Application core

- app/main.py - FastAPI app, router wiring, root message

- app/com

Routes

- app/routes/analyze.py - Orchestrates validate -> encode -> VLM -> report -> save

- app/rout

Services

- app/services/vision_service.py - OpenAI integration

- app/serv

Utilities

- app/utis/image_utils.py - Content-type gate + base64

Policy context

- context_docs/warehouse_safety_rules.md -> warehouse_scene

- context_

Tests, ops, docs

- tests/* - pytest suite

- requirem

6. Current strengths

1. Clear layered architecture (routes -> services -> schemas)

2. Typed c

7. Current limitations / gaps

Gap | Current state

Real databa

8. Recommended next steps

Tier 1 - Highest ROI, lowest risk

1. GitHub Actions CI (pytest on push/PR)

2. Structure

Tier 2 - Strong portfolio signal

6. SQLite + SQLAlchemy for reports

7. Simple A

Tier 3 - Only if role-specific

- Async queue, Terraform, microservices split, agent frameworks - defer until baseline eval + deploy exist

9. Do not over-engineer

- Keep single FastAPI service until scale demands split

- Prefer S

Summary

This is a well-structured multimodal API prototype: VLM structured extraction, policy grounding, escalation reporting, file persistence, Docker, and tests. It does not yet include auth, real DB, vector RAG, async processing, CI/CD, cloud deploy, observability, or evals. Section 8 prioritizes next steps without unnecessary enterprise complexity.